Code Explanation

Test Data Processing Code Explanation

This code is a unit test for the `clean_data` function, which is used to clean customer and order data using Apache Spark. The test class `TestDataProcessing` contains methods to set up a Spark session, create test data, and test the `clean_data` function.

Overview of the Code
The code is designed to test the functionality of the `clean_data` function, which cleans customer and order data by removing duplicates and null values, and calculating the total amount for each order.

Step 1: Setting Up the Spark Session and Test Data
The `setUpClass` method is used to create a Spark session and test data for customer and order information. This includes creating DataFrames with sample data and defining their schemas.

```
@classmethod
def setUpClass(cls):
    cls.spark = SparkSession.builder
        .appName("TestDataProcessing")
        .master("local[1]")
        .getOrCreate()

    customer_data = [
        ("C001", "John Doe", "john@example.com", "North"),
        ("C002", "Jane Smith", "jane@example.com", "South"),
        ("C003", "Bob Johnson", "bob@example.com", "East"),
        ("C003", "Bob Johnson", "bob@example.com", "East"),
        ("C004", None, "alice@example.com", "West"),
        ("C005", "Charlie Brown", "charlie@example.com", "North")
    ]

    customer_schema = StructType([
        StructField("CustId", StringType(), True),
        StructField("Name", StringType(), True),
        StructField("EmailId", StringType(), True),
        StructField("Region", StringType(), True)
    ])

    cls.customer_df = cls.spark.createDataFrame(customer_data, schem
a=customer_schema)
```

Step 2: Defining the Test Data for Orders
The `setUpClass` method also creates test data for orders, including duplicate and null values.

```
order_data = [
    ("O001", "Laptop", 1000.0, 2, datetime.date(2023, 1, 15), "C001"
),
    ("O002", "Phone", 500.0, 1, datetime.date(2023, 2, 20), "C002"),
    ("O003", "Tablet", 300.0, 3, datetime.date(2023, 3, 10), "C003")
,
    ("O003", "Tablet", 300.0, 3, datetime.date(2023, 3, 10), "C003")
,
    ("O004", "Monitor", None, 2, datetime.date(2023, 4, 5), "C001"),
```

```
    ("O005", "Keyboard", 50.0, 5, datetime.date(2023, 5, 12), "C005"
)
]

order_schema = StructType([
    StructField("OrderId", StringType(), True),
    StructField("ItemName", StringType(), True),
    StructField("PricePerUnit", DoubleType(), True),
    StructField("Qty", IntegerType(), True),
    StructField("Date", DateType(), True),
    StructField("CustId", StringType(), True)
])

cls.order_df = cls.spark.createDataFrame(order_data, schema=order_sc
hema)
```

Step 3: Stopping the Spark Session
The `tearDownClass` method is used to stop the Spark session after the tests are completed.

```
@classmethod
def tearDownClass(cls):
    cls.spark.stop()
```

Step 4: Testing the clean_data Function
The `test_clean_data` method tests the `clean_data` function by checking if it correctly removes duplicates and null values, and calculates the total amount for each order.

```
def test_clean_data(self):
    cleaned_customer_df, cleaned_order_df = clean_data(self.customer
_df, self.order_df)

    self.assertEqual(cleaned_customer_df.count(), 4)

    self.assertEqual(cleaned_order_df.count(), 4)

    order_with_total = cleaned_order_df.filter(cleaned_order_df.Orde
rId == "O001").collect()[0]
    self.assertEqual(order_with_total.TotalAmount, 2000.0)

    self.assertEqual(cleaned_customer_df.filter(cleaned_customer_df.
Name.isNull()).count(), 0)
    self.assertEqual(cleaned_order_df.filter(cleaned_order_df.PriceP
erUnit.isNull()).count(), 0)
```

Step 5: Running the Unit Tests
The `unittest.main()` function is used to run the unit tests.

```
if __name__ == "__main__":
    unittest.main()
```